



Міністерство освіти і науки України
Національний технічний університет України
“Київський політехнічний інститут імені Ігоря Сікорського”
Факультет інформатики та обчислювальної техніки
Кафедра інформаційних систем та технологій

Технології розроблення програмного забезпечення
Лабораторна робота №5
«Шаблони «Adapter », «Builder », «Command»,
«Chain of Responsibility», «Prototype».»
Варіант - 4

Виконала:
Студентка групи ІА-22
Третякова Ельвіра

Перевірив:
Мягкий Михайло Юрійович

Київ 2024

Зміст

Короткі теоретичні відомості.....	3
Реалізація не менше 3-х класів відповідно до обраної теми.....	8
Реалізація шаблону за обраною темою.....	10
<i>Діаграма класів для паттерну Proxy.....</i>	<i>15</i>
<i>Проблема, яку допоміг вирішити шаблон Prototype</i>	<i>16</i>
<i>Переваги застосування паттерну Prototype</i>	<i>17</i>
<i>Посилання на GitHub</i>	<i>18</i>
Висновок	18

Тема: Шаблони «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype»

Мета: Ознайомитися з шаблонами проектування «Adapter», «Builder», «Command», «Chain of responsibility», «Prototype» та набутти практичних навичок їх застосування. Реалізувати частину функціоналу програми за допомогою одного з розглянутих шаблонів для досягнення конкретних функціональних можливостей та забезпечення ефективної взаємодії між класами.

Хід роботи

Варіант – 4: Графічний редактор (proxy, prototype, decorator, bridge, flyweight, SOA)
Графічний редактор повинен вміти створювати / редагувати растрові (або векторні на розсуд студента) зображення в 2-3 основних популярних форматах (bmp, png, jpg), мати панель інструментів для створення графічних примітивів, вибору кольорів, нанесення тексту, додавання найпростіших візуальних ефектів (ч/б растр, інфрачервоний растр, 2-3 на вибір учня), роботи з шарами.

Короткі теоретичні відомості

«Анти-шаблони» проектування

Анти-патерни (anti-patterns), також відомі як пастки (pitfalls) - це класи найбільш часто впроваджуваних поганих рішень проблем. Вони вивчаються, як категорія, в разі коли їх хочуть уникнути в майбутньому, і деякі їхні окремі випадки можуть бути розпізнані при вивченні непрацюючих систем.

Термін походить з інформатики, від авторів «Банди чотирьох» книги «Шаблони проектування», яка заклала приклади практики хорошого програмування. Автори назвали ці хороші методи «шаблонами проектування», і протилежними їм є «анти-патерни».

Частиною хоршої практики програмування є уникнення анти-патернів.

Управління розробкою ПЗ:

- Дим і дзеркала: Демонстрація неповних функцій.
- Роздування ПЗ: Збільшення вимог до ресурсів у наступних версіях.
- Функції для галочки: Наявність непов'язаних функцій для реклами.

Розробка ПЗ:

- Інверсія абстракції: Приховування частини функціоналу.
- Невизначена точка зору: Моделі без чіткої специфікації.
- Великий клубок бруду: Система без чіткої структури.
- Бензинова фабрика: Необов'язкова складність дизайну.
- Затичка на введення даних: Недостатня обробка неправильного введення.

ООП:

- Базовий клас-утиліта: Спадкування замість делегування.
- Божественний об'єкт: Надмірна концентрація функцій в одному класі.
- Самотність: Надмірне використання патерну Singleton.

Анти-патерни в програмуванні:

- Непотрібна складність: Ускладнення рішень без необхідності.
- Дія на відстані: Несподівані взаємодії між різними частинами системи.
- Накопичити і запустити: Використання глобальних змінних для параметрів.
- Сліпа віра: Недостатня перевірка результатів або виправлень.
- Активне очікування: Використання ресурсів під час очікування події замість асинхронного підходу.
- Кешування помилки: Забуте скидання прапора після обробки помилки.

- Перевірка типу замість інтерфейсу: Перевірка типу замість використання інтерфейсу.
- Кодування шляхом виключення: Обробка випадків через винятки.
- Потік лави: Залишення поганого коду через високу вартість його видалення.
- Спагеті-код: Заплутаний код з некерованим виконанням.

Методологічні анти-патерни:

- Програмування методом копіювання-вставки: Копіювання коду замість створення спільних рішень.
- Дефакторинг: Заміна функціональності документацією.
- Золотий молоток: Впевненість у застосуванні одного рішення для всіх проблем.
- Передчасна оптимізація: Оптимізація без достатньої інформації.
- Винахід колеса: Створення нових рішень замість використання існуючих.
- Винахід квадратного колеса: Погане рішення, коли є хороше.
- Самознищення: Помилка, що призводить до фатальної поведінки програми.

Анти-патерни управління конфігурацією:

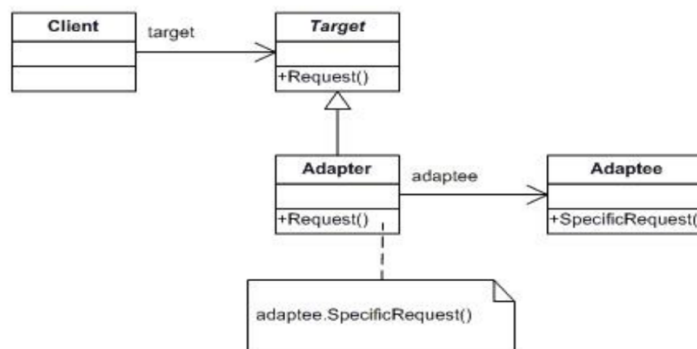
- DLL-пекло: Проблеми з версіями і доступністю DLL.
- Пекло залежностей: Проблеми з версіями залежних продуктів.

Організаційні анти-патерни:

- Аналітичний параліч: Надмірний аналіз без реалізації.
- Дійна корова: Неінвестування в розвиток продукту.
- Тривале старіння: Перенесення зусиль на портинг замість нових рішень.
- Скидування витрат: Перенесення витрат на інші відділи.
- Повзуче покращення: Додавання змін, що знижують загальну якість.
- Розробка комітетом: Розробка без централізованого керівництва.
- Ескалація зобов'язань: Продовження хибного рішення.
- Розповзання рамок: Втрата контролю над проектом.
- Замикання на продавці: Залежність від одного постачальника.

Шаблон «Adapter»

Структура:



Призначення патерну:

Шаблон "adapter" (адаптер) використовується для адаптації інтерфейсу одного об'єкту до іншого. Наприклад, існує декілька бібліотек для роботи з принтерами, проте кожна має різний інтерфейс (хоча однакові можливості і призначення). Має сенс розробити уніфікований інтерфейс (сканування, асинхронне сканування, двостороннє сканування, потокове сканування і тому подібне), і реалізувати відповідні адаптери для приведення бібліотек до уніфікованого інтерфейсу. Це дозволить в програмі звертатися до загального

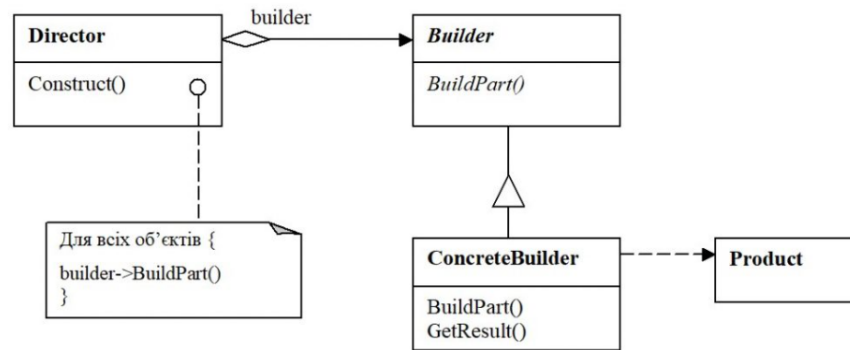
інтерфейсу, а не приводити різні сценарії роботи залежно від способу реалізації бібліотеки. Адаптери також називаються "wrappers" (обгортками).

Переваги та недоліки:

- + Відокремлює та приховує від клієнта подробиці перетворення різних інтерфейсів.
- Ускладнює код програми внаслідок введення додаткових класів.

Шаблон «Builder»

Структура:



Призначення патерну:

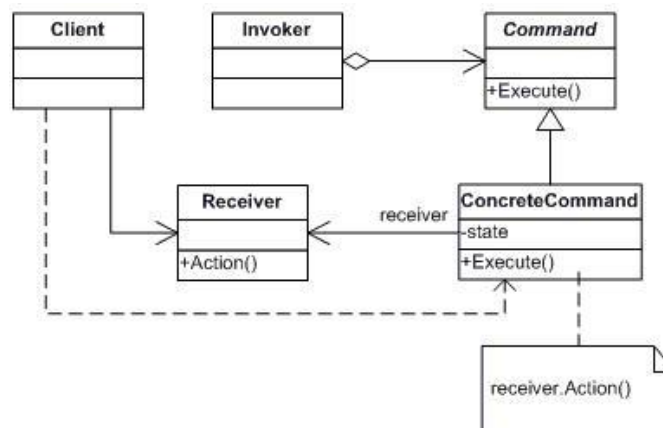
Шаблон "builder" (будівельник) використовується для відділення процесу створення об'єкту від його представлення. Це доречно у випадках, коли об'єкт має складний процес створення (наприклад, Web- сторінка як елемент повної відповіді web- сервера) або коли об'єкт повинен мати декілька різних форм створення (наприклад, при конвертації тексту з формату у формат).

Переваги та недоліки:

- + Дозволяє використовувати один і той самий код для створення різноманітних продуктів.
- Клієнт буде прив'язаний до конкретних класів будівельників, тому що в інтерфейсі будівельника може не бути методу отримання результату.

Шаблон «Command»

Структура:



Призначення патерну:

Шаблон "command" (команда) перетворить звичайний виклик методу в клас. Таким чином дії в системі стають повноправними об'єктами. Це зручно в наступних випадках:

- Коли потрібна розвинена система команд - відомо, що команди будуть додаватися;
- Коли потрібна гнучка система команд - коли з'являється необхідність додавати командам

можливість відміни, логування і інш.;

- Коли потрібна можливість складання ланцюжків команд або виклику команд в певний час; Об'єкт команда сама по собі не виконує ніяких фактичних дій окрім перенаправлення запиту одержувачеві (тобто команди все ж виконуються одержувачем), однак ці об'єкти можуть зберігати дані для підтримки додаткових функцій відміни, логування і інш. Наприклад, команда вставки символу може запам'ятовувати символ, і при виклику відміни викликати відповідну функцію витирання символу. Можна також визначити параметр "застосовності" команди (наприклад, на картинці писати не можна) - і використати цей атрибут для засвічування піктограми в меню.

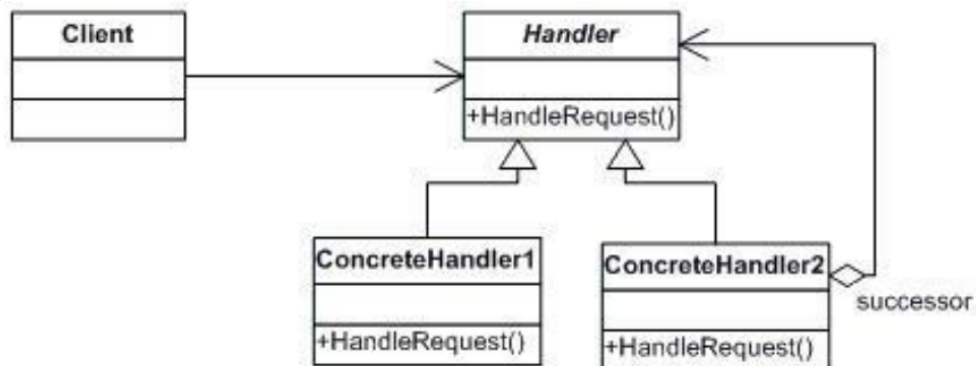
Такий підхід до команд дозволяє побудувати дуже гнучку систему команд, що настроюється. У більшості додатків це буде зайвим (використовується спрощений варіант), проте життєво важливий в додатках з великою кількістю команд (редактори).

Переваги та недоліки:

- + Прибирає пряму залежність між об'єктами, що викликають операції, та об'єктами, які їх безпосередньо виконують.
- + Дозволяє реалізувати просте скасування і повтор операцій.
- + Дозволяє реалізувати відкладений запуск операцій.
- + Дозволяє збирати складні команди з простих.
- + Реалізує принцип відкритості/закритості.
- Ускладнює код програми внаслідок введення великої кількості додаткових класів.

Шаблон «Chain of Responsibility»

Структура:



Призначення патерну:

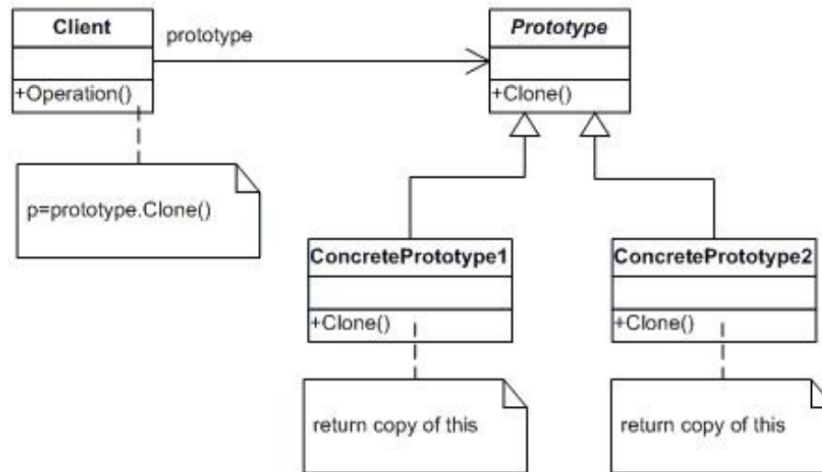
Шаблон "chain of responsibility" (ланцюг відповідальності) частково можна спостерігати в житті, коли підписання відповідного документу проходить від його складання у одного із співробітників компанії через менеджера і начальника до головного начальника, який ставить свій підпис.

Переваги та недоліки:

- + Зменшує залежність між клієнтом та обробниками.
- + Реалізує принцип єдиного обов'язку.
- + Реалізує принцип відкритості/закритості.
- Запит може залишитися ніким не опрацьованим.

Шаблон «Prototype»

Структура:



Призначення патерну:

Шаблон "prototype" (прототип) використовується для створення об'єктів за "шаблоном" (чи "кресленню", "ескізу") шляхом копіювання шаблонного об'єкту. Для цього визначається метод "клонувати" в об'єктах цього класу.

Цей шаблон зручно використати, коли заздалегідь відомо як виглядатиме кінцевий об'єкт (мінімізується кількість змін до об'єкту шляхом створення шаблону), а також для видалення необхідності створення об'єкту - створення відбувається за рахунок клонування, і зухвалій програмі абсолютно немає необхідності знати, як створювати об'єкт.

Також, це дозволяє маніпулювати об'єктами під час виконання програми шляхом налаштування відповідних шаблонів; значно зменшується ієрархія спадкоємства (оскільки в іншому випадку це були б не шаблони, а вкладені класи, що наслідують).

Переваги та недоліки:

- + Дозволяє клонувати об'єкти без прив'язки до їхніх конкретних класів.
- + Менша кількість повторювань коду ініціалізації об'єктів.
- + Прискорює створення об'єктів.
- + Альтернатива створенню підкласів під час конструювання складних об'єктів.
- Складно клонувати складові об'єкти, що мають посилання на інші об'єкти.

Реалізація не менше 3-х класів відповідно до обраної теми

Структура проекту з реалізованими класами зображена на рисунку 1:

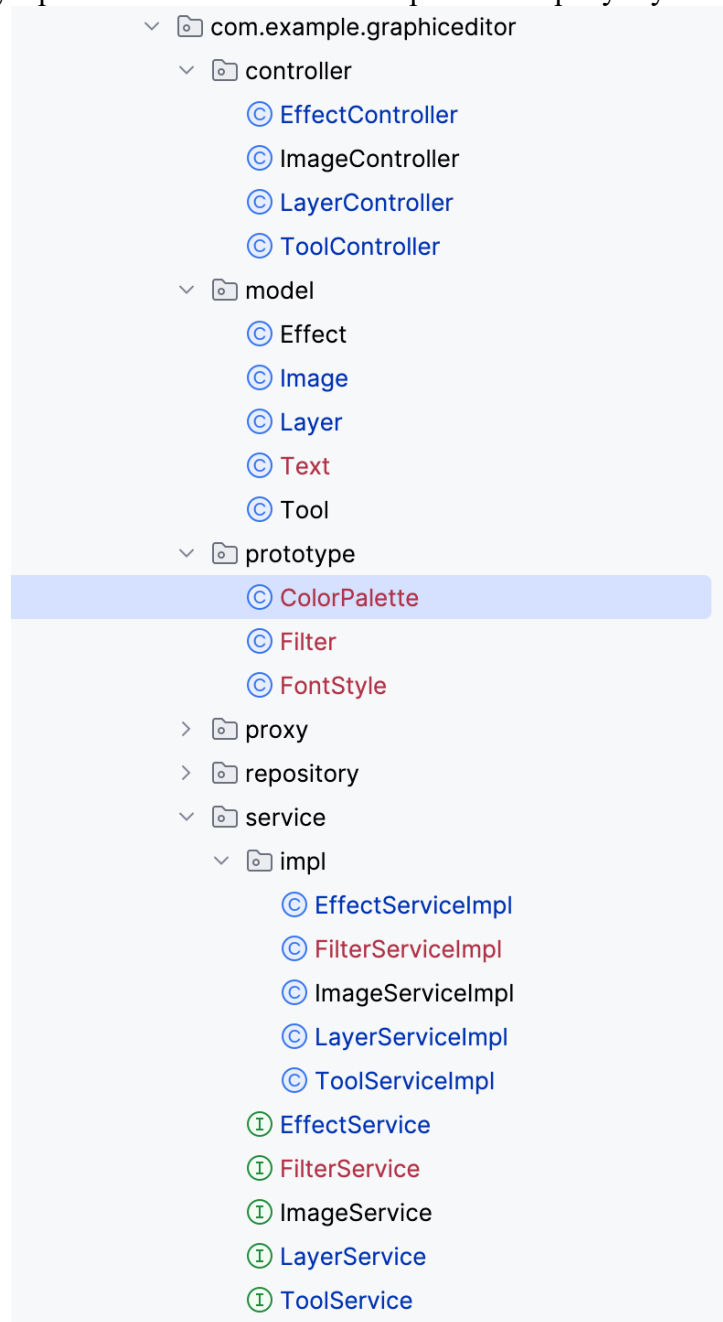


Рис.1. Структура проекту з реалізованими класами

У ході лабораторної роботи була реалізована система графічного редактора, в якій використано патерн Prototype для роботи з об'єктами, що часто повторюються та потребують створення копій. Основна мета — спростити процес створення нових об'єктів шляхом копіювання існуючих, замість ініціалізації їх з нуля. Це дозволяє суттєво оптимізувати код і підвищити продуктивність системи.

Для реалізації патерну Prototype було додано такі класи у відповідну папку `prototype`:

- ColorPalette — клас для збереження палітри кольорів, яка використовується в графічному редакторі.
- Filter — клас для представлення фільтрів, що застосовуються до зображень.
- FontStyle — клас для роботи зі стилями шрифтів (назва шрифту, розмір, жирність, курсив).

Також для зберігання текстових даних у редакторі було додано клас Text у папку model. Він відповідає за управління стилем тексту, що включає назву шрифту, розмір, жирність та стиль (курсив/звичайний).

Система організована таким чином, що всі класи, які реалізують Prototype, можуть бути клоновані за допомогою методу clone(), що дозволяє швидко створювати копії об'єктів, які зберігають однакові властивості.

Додатково було реалізовано базову функціональність графічного редактора:

- Обробка зображень за допомогою фільтрів.
- Застосування палітри кольорів.
- Робота зі стилями шрифту для текстових шарів.

Завдяки використанню патерну Prototype, система стала більш гнучкою, оскільки дозволяє швидко та легко створювати копії різних графічних об'єктів. Це особливо важливо для роботи з об'єктами, які мають складну структуру або великі обсяги даних.

Також було додано клас Text:

Клас створено для зберігання та налаштування тексту в графічному редакторі. Він зберігає такі властивості, як назва шрифту, розмір, жирність та стиль.

Основні функції:

- Зміна параметрів тексту через відповідні методи.
- Збереження стилістичних властивостей тексту.

Сервіси для роботи із фільтрами

Для забезпечення зручної роботи з фільтрами було реалізовано окремий пакет service, який включає сервіси для управління об'єктами фільтрів.

1. Інтерфейс FilterService:

Інтерфейс визначає основні методи для роботи з фільтрами:

- Створення фільтра – дозволяє створювати новий фільтр.
- Отримання фільтра за ID – повертає фільтр на основі його ідентифікатора.
- Отримання всіх фільтрів – повертає список усіх створених фільтрів.
- Оновлення фільтра – дозволяє оновлювати параметри фільтра.
- Видалення фільтра – видаляє фільтр за ідентифікатором.
- Клонування фільтра – створює копію фільтра з існуючим ID.

2. Реалізація FilterService (клас FilterServiceImpl)

Клас реалізує інтерфейс FilterService і працює з репозиторієм фільтрів. Використовує Spring Framework для впровадження залежностей та управління даними.

Основні методи:

- createFilter(Filter filter): Зберігає новий фільтр у репозиторії.
- getFilterById(int id): Отримує фільтр за ідентифікатором, кидає помилку, якщо фільтр не знайдено.
- getAllFilters(): Повертає список усіх фільтрів із бази.
- updateFilter(int id, Filter filter): Оновлює дані фільтра за його ID.
- deleteFilter(int id): Видаляє фільтр, якщо він існує.
- cloneFilter(int id): Використовує метод clone() для створення копії існуючого фільтра.

Переваги:

- Простота використання сервісу для клієнтського коду.
- Підтримка CRUD-операцій із фільтрами.
- Застосування патерна Prototype для клонування.

Реалізація шаблону за обраною темою

У ході роботи було реалізовано систему графічного редактора, в якій застосовано патерн Prototype для забезпечення ефективного створення об'єктів, що часто повторюються. Основна мета використання цього патерна — оптимізація роботи з об'єктами, які потребують копіювання, шляхом використання методу clone() замість повного створення нового екземпляра. Це значно покращує продуктивність та спрощує код.

Опис реалізації:

1. Клас ColorPalette

Клас ColorPalette призначений для роботи з палітрою кольорів. Він дозволяє зберігати, додавати, видаляти кольори, а також підтримує клонування палітри для створення її копії.

Основні функції:

1. Зберігання кольорів: Палітра представлена у вигляді списку кольорів, які можна змінювати (додавати чи видаляти).
2. Інкапсуляція: Доступ до списку кольорів здійснюється через геттери та сеттери.
3. Клонування: Реалізовано можливість клонування палітри, що дозволяє створювати її копію зі збереженням стану.

Основні можливості:

- Палітра є змінною, оскільки кольори можна додавати і видаляти.
- Метод clone() дозволяє створювати копію об'єкта без ризику зміни оригінального об'єкта.
- Захист від небажаних змін: при встановленні нового списку використовується копіювання.

Переваги:

- Легка підтримка і розширення палітри кольорів.
- Гнучкість при роботі з копіями палітри.

```
© ColorPalette.java x
1 package com.example.graphiceditor.prototype;
2
3 import java.util.ArrayList;
4 import java.util.List;
5
6 public class ColorPalette implements Cloneable { 9 usages
7     private List<String> colors; 8 usages
8
9     public ColorPalette(List<String> colors) { no usages
10         this.colors = new ArrayList<>(colors); // Уникаємо мутації
11     }
12
13     public List<String> getColors() { 1 usage
14         return colors;
15     }
16
17     public void setColors(List<String> colors) { 1 usage
18         this.colors = new ArrayList<>(colors);
19     }
20
21     public void addColor(String color) { no usages
22         colors.add(color);
23     }
24
25     public void removeColor(String color) { no usages
26         colors.remove(color);
27     }
28
29     // Реалізація методу clone
30     @Override
31     public ColorPalette clone() {
32         try {
33             ColorPalette cloned = (ColorPalette) super.clone();
34             cloned.colors = new ArrayList<>(this.colors); // Копіюємо список кольорів
35             return cloned;
36         } catch (CloneNotSupportedException e) {
37             throw new RuntimeException("Error cloning ColorPalette object", e);
38         }
39     }
40
41     @Override
42     public String toString() {
43         return "ColorPalette{colors=" + colors + "}";
44     }
45 }
```

Рис.2 Реалізація класу ColorPalette

2. Клас Filter

Клас Filter моделює графічний фільтр, який можна застосовувати до зображень. Кожен фільтр має назву, інтенсивність і можливість клонування для повторного використання.

Основні функції:

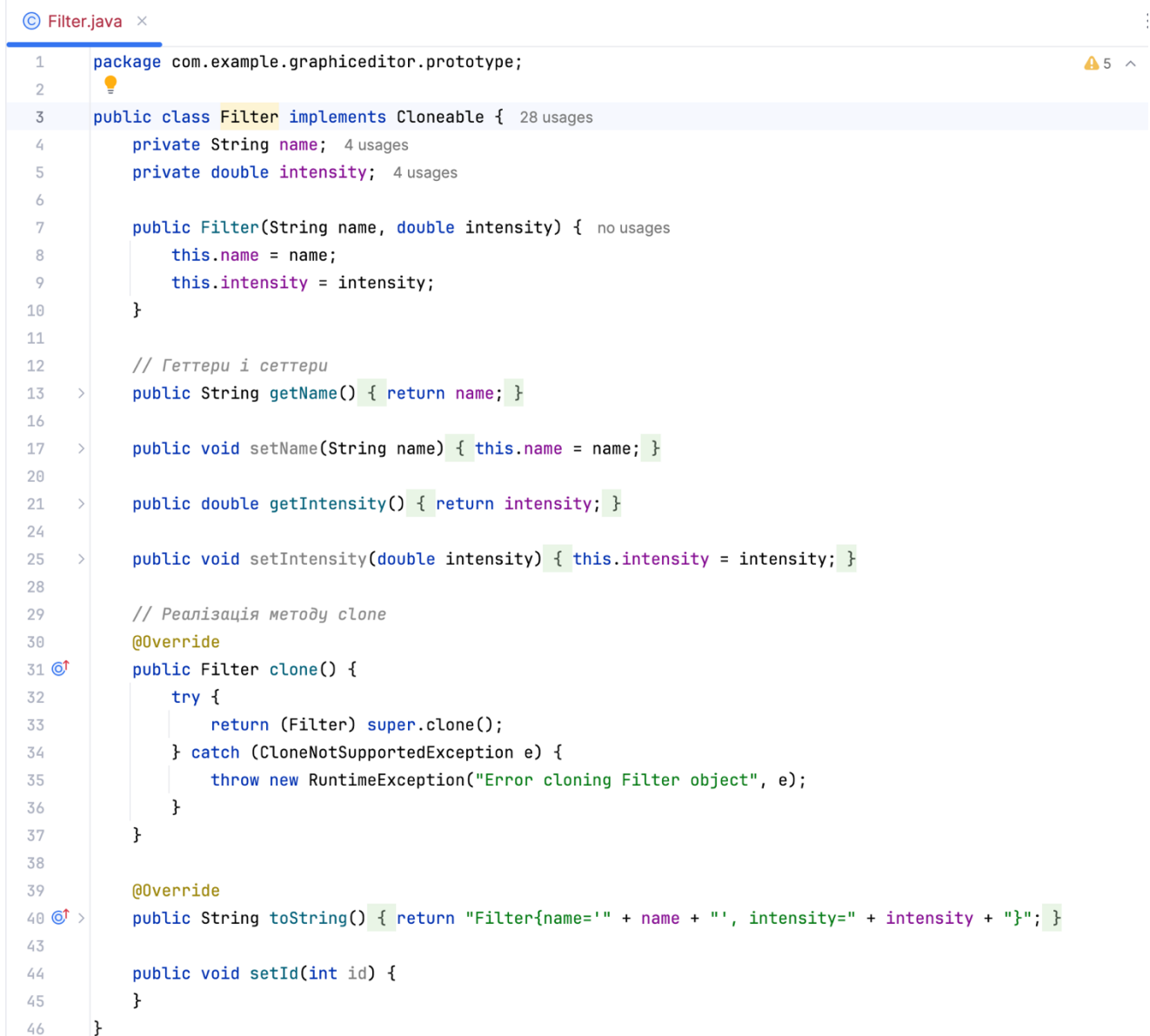
1. Зберігання інформації про фільтр: Кожен фільтр має назву (наприклад, "Blur", "Contrast") та параметр інтенсивності (число від 0 до 1).
2. Модифікація фільтра: Можна змінювати назву та інтенсивність.
3. Клонування: Реалізована можливість створювати копії фільтрів.

Основні можливості:

- Фільтр має параметр `intensity`, що дозволяє змінювати ступінь впливу фільтра на зображення.
- Метод `clone()` дозволяє дублювати фільтр для повторного використання.

Переваги:

- Простота налаштування параметрів фільтра.
- Клонування дозволяє використовувати копії фільтрів у різних контекстах.



```

1  package com.example.graphiceditor.prototype;
2
3  public class Filter implements Cloneable { 28 usages
4      private String name; 4 usages
5      private double intensity; 4 usages
6
7      public Filter(String name, double intensity) { no usages
8          this.name = name;
9          this.intensity = intensity;
10     }
11
12     // Геттери і сеттери
13     public String getName() { return name; }
14
15
16
17     public void setName(String name) { this.name = name; }
18
19
20
21     public double getIntensity() { return intensity; }
22
23
24
25     public void setIntensity(double intensity) { this.intensity = intensity; }
26
27
28
29     // Реалізація методу clone
30     @Override
31     public Filter clone() {
32         try {
33             return (Filter) super.clone();
34         } catch (CloneNotSupportedException e) {
35             throw new RuntimeException("Error cloning Filter object", e);
36         }
37     }
38
39     @Override
40     public String toString() { return "Filter{name='" + name + "', intensity=" + intensity + "}"; }
41
42
43
44     public void setId(int id) {
45     }
46 }

```

Рис.3 Реалізація класу Filter

3. Клас *FontStyle*

Клас *FontStyle* моделює стиль шрифту для текстових елементів. Він зберігає інформацію про назву шрифту, його розмір, а також параметри *bold* і *italic*.

Основні функції:

1. Зберігання параметрів шрифту: Шрифт має назву, розмір, і може бути жирним або курсивним.

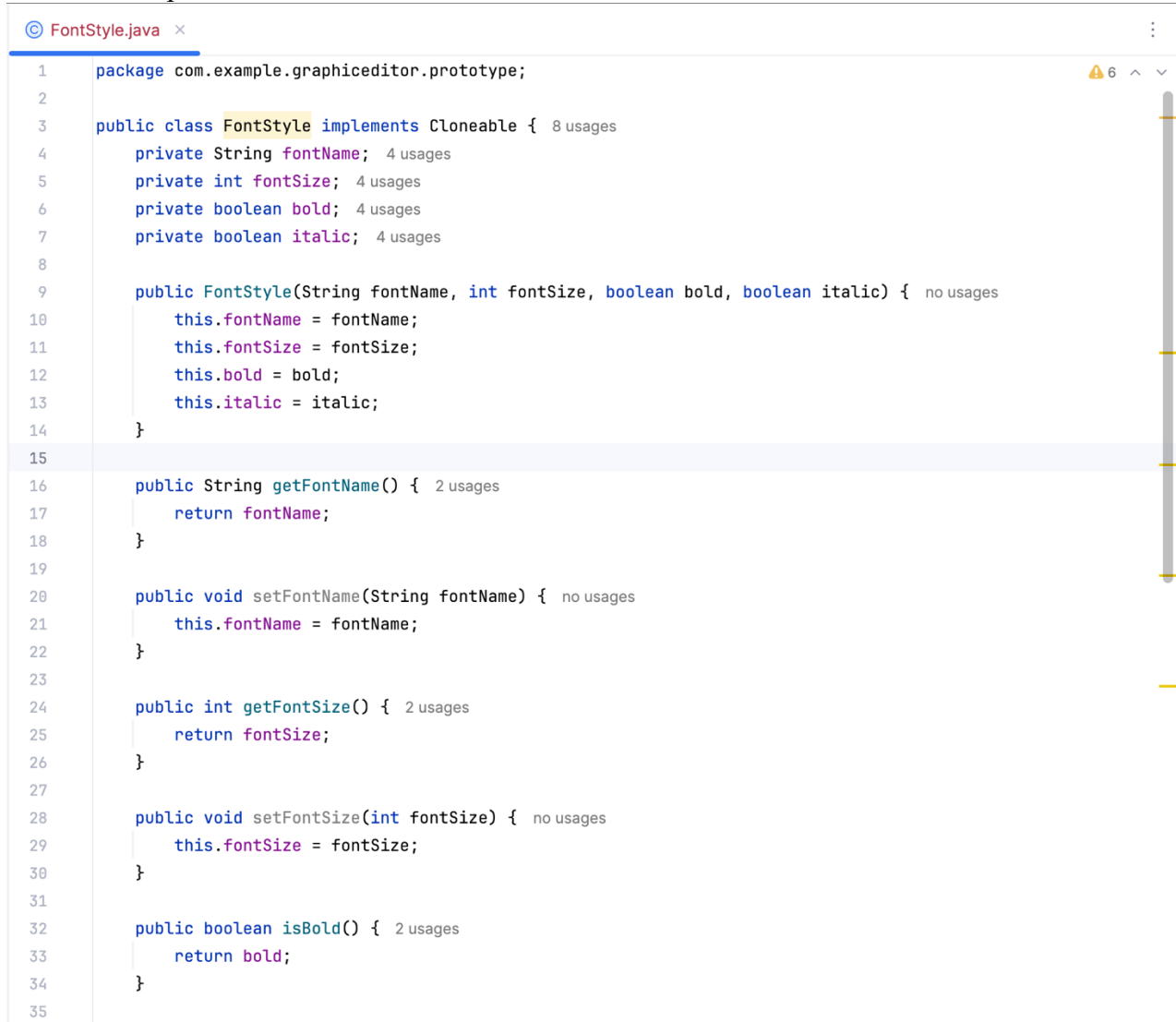
2. Модифікація стилю: Доступ до всіх параметрів здійснюється через геттери й сеттери.
3. Клонування: Реалізована можливість дублювання стилів шрифту.

Основні можливості:

- Клас дозволяє створювати складні стилі шрифту з різними параметрами.
- Можна зручно копіювати стиль шрифту через метод **clone()**.

Переваги:

- Простота зміни параметрів стилю.
- Використання копій дозволяє застосовувати стилі до різних текстових елементів без повторної ініціалізації.



```
1 package com.example.graphiceditor.prototype;
2
3 public class FontStyle implements Cloneable { 8 usages
4     private String fontName; 4 usages
5     private int fontSize; 4 usages
6     private boolean bold; 4 usages
7     private boolean italic; 4 usages
8
9     public FontStyle(String fontName, int fontSize, boolean bold, boolean italic) { no usages
10         this.fontName = fontName;
11         this.fontSize = fontSize;
12         this.bold = bold;
13         this.italic = italic;
14     }
15
16     public String getFontName() { 2 usages
17         return fontName;
18     }
19
20     public void setFontName(String fontName) { no usages
21         this.fontName = fontName;
22     }
23
24     public int getFontSize() { 2 usages
25         return fontSize;
26     }
27
28     public void setFontSize(int fontSize) { no usages
29         this.fontSize = fontSize;
30     }
31
32     public boolean isBold() { 2 usages
33         return bold;
34     }
35 }
```

```

36     public void setBold(boolean bold) { no usages
37         this.bold = bold;
38     }
39
40     public boolean isItalic() { 2 usages
41         return italic;
42     }
43
44     public void setItalic(boolean italic) { no usages
45         this.italic = italic;
46     }
47
48     // Реалізація методу clone
49     @Override
50     public FontStyle clone() {
51         try {
52             return (FontStyle) super.clone();
53         } catch (CloneNotSupportedException e) {
54             throw new RuntimeException("Error cloning FontStyle object", e);
55         }
56     }
57
58     @Override
59     public String toString() {
60         return "FontStyle{fontName='" + fontName + "', fontSize=" + fontSize +
61             ", bold=" + bold + ", italic=" + italic + "}";
62     }
63 }

```

Рис.4 Реалізація класу FontStyle

Класи ColorPalette, Filter і FontStyle реалізують патерн Prototype, що дозволяє ефективно створювати копії об'єктів. Це спрощує роботу з графічним редактором і забезпечує його модульність.

Діаграма класів для паттерну Proxu

Діаграма класів, яка реалізовує патерн Prototype зображена на рисунку 5:

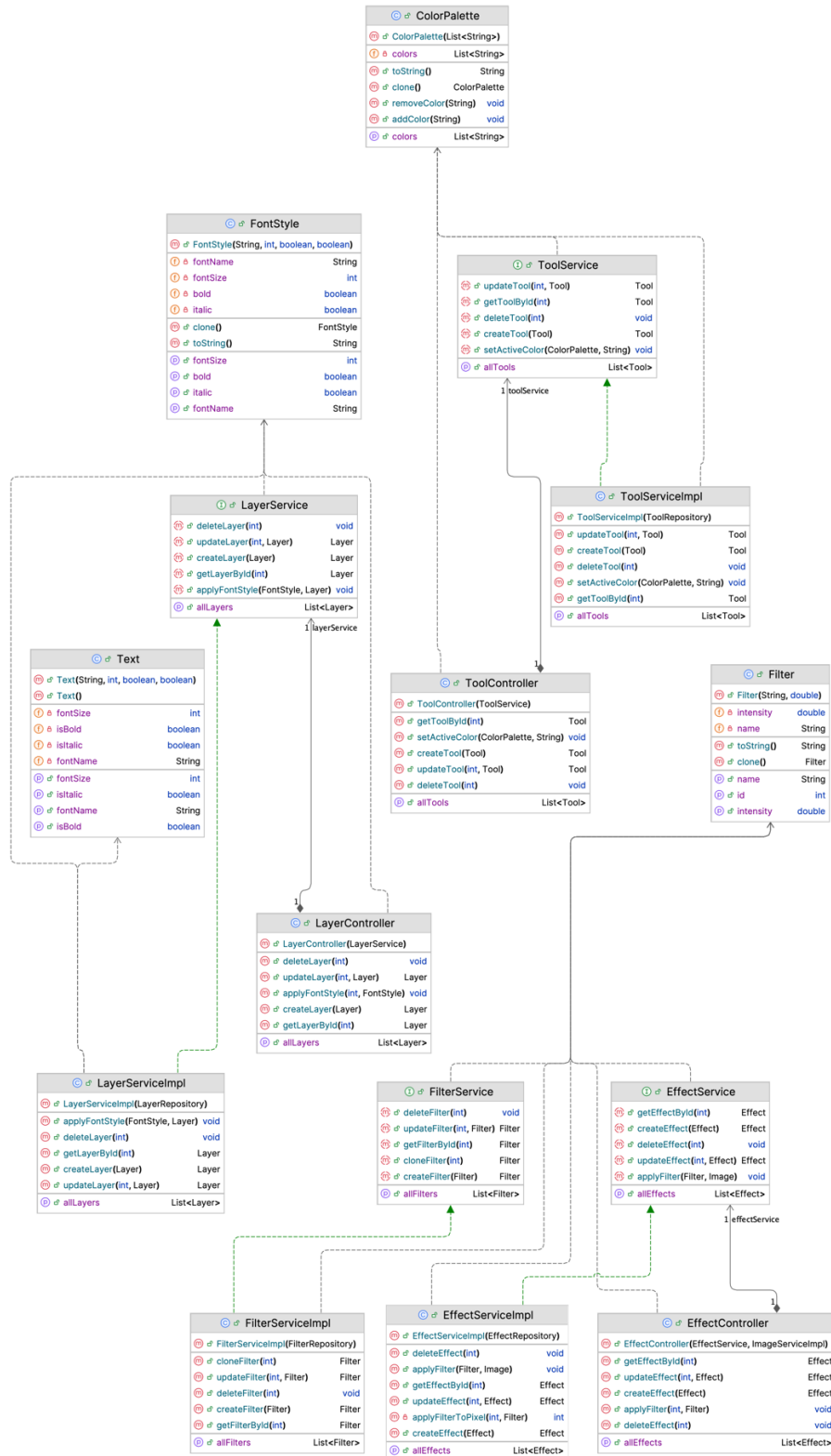


Рис. 5 Діаграма класів для паттерну Prototype

Ця діаграма класів представляє структуру застосування шаблону Prototype для роботи з графічними елементами в межах графічного редактора.

Основні класи:

1. **FontStyle**

Представляє стилі тексту (шрифти, розмір, жирність, курсив).

Методи:

clone(): створює копію об'єкта стилю.

toString(): повертає текстове представлення об'єкта.

2. **ColorPalette**

Зберігає кольори у вигляді списку.

Методи:

clone(): створює копію палітри.

addColor(String), removeColor(String): керування кольорами.

3. **Filter**

Містить інформацію про фільтри (назва, інтенсивність).

Методи:

clone(): створює копію фільтра.

toString(): повертає опис фільтра.

4. **Text**

Представляє текстовий елемент.

Наслідується від FontStyle.

Зв'язки між класами:

1. Prototype реалізовано через метод clone() в основних класах (FontStyle, ColorPalette, Filter), що дозволяє створювати нові об'єкти шляхом копіювання.
2. Контролери працюють із відповідними сервісами для забезпечення користувацьких дій.
3. Сервіси зберігають і керують списками об'єктів, реалізуючи їхнє створення, редагування та видалення.

Проблема, яку допоміг вирішити шаблон Prototype

Шаблон Prototype допоміг вирішити кілька важливих проблем у моєму графічному редакторі:

1. Швидке створення об'єктів із подібними властивостями

Замість створення нових об'єктів із нуля і повторного налаштування їхніх параметрів, можемо швидко отримувати копії вже існуючих об'єктів за допомогою методу clone().

Наприклад:

- Копіювання палітри кольорів (ColorPalette) для створення нових варіантів із мінімальними змінами.
- Клонування фільтра (Filter) для застосування різних рівнів інтенсивності.
- Клонування стилю шрифту (FontStyle) для створення текстових елементів із подібними налаштуваннями.

2. Уникнення мутації спільних даних

При клонуванні складних об'єктів, таких як ColorPalette, створюється нова копія списку кольорів, завдяки чому зміни в копії не впливають на оригінальний об'єкт. Це особливо важливо для роботи з шарами, фільтрами та ефектами, де кожен елемент має свою незалежну конфігурацію.

3. Інкапсуляція логіки копіювання

Метод clone() автоматизує процес копіювання об'єктів, приховуючи складнощі реалізації. Не потрібно вручну дублювати поля чи заново ініціалізувати кожен елемент.

4. Зниження витрат пам'яті та обчислень

При створенні копій складних об'єктів (наприклад, палітри кольорів або стилів шрифтів) шаблон Prototype дозволяє уникати перевантаження процесора, адже копіюються тільки необхідні частини даних, без дублювання зайвих операцій.

ColorPalette

- Метод clone() створює копію об'єкта та його списку кольорів. Це дозволяє безпечно експериментувати з палітрами без ризику зіпсувати оригінал.

Filter

- Завдяки clone() можна створювати нові фільтри з однаковими базовими параметрами, але з різними значеннями інтенсивності або назвами.

FontStyle

- clone() дозволяє копіювати стиль шрифтів, наприклад, для створення текстових шарів із подібним оформленням.

Шаблон Prototype оптимізував процес створення нових об'єктів, зробив систему більш гнучкою, зменшив ризики неочікуваних змін даних та прискорив розробку складних елементів графічного редактора.

Переваги застосування патерну Prototype

Використання патерну Prototype у моєму графічному редакторі має такі переваги:

1. Швидке створення копій об'єктів

- У графічному редакторі часто потрібно створювати об'єкти з подібними властивостями, наприклад, палітри кольорів, фільтри чи стилі шрифтів. Завдяки патерну Prototype нові екземпляри можна отримати швидко, копіюючи вже існуючі об'єкти.

2. Уникнення мутації даних

- Клонування об'єктів гарантує, що зміни в копії не впливають на оригінальний об'єкт. Наприклад, при роботі з палітрою кольорів (ColorPalette) або налаштуваннями фільтрів (Filter) кожна копія є незалежною, що запобігає конфліктам у проекті.

3. Ефективна робота з шаблонами

- Prototype дозволяє зберігати шаблони часто використовуваних елементів (наприклад, стилі шрифтів чи стандартні фільтри) та швидко створювати їх копії для використання у різних частинах проекту.

4. Оптимізація ресурсів

- Завдяки клонуванню можна уникнути витратних операцій повторного створення складних об'єктів, таких як палітри чи багат шарові налаштування, економлячи час і ресурси.

5. Зручне налаштування копій

- Після створення копії об'єкта його легко модифікувати для конкретних потреб. Наприклад, у Filter можна змінити інтенсивність, не зачіпаючи базові параметри, або в FontStyle налаштувати розмір шрифту та стилізацію для різних текстових елементів.

6. Розширюваність

- Патерн Prototype спрощує додавання нових типів об'єктів у редакторі. Новий об'єкт лише має реалізувати метод clone(), що робить систему більш гнучкою та масштабованою.

Ці переваги зробили графічний редактор більш ефективним, забезпечивши простоту створення та обробки незалежних об'єктів у великих проектах.

Посилання на GitHub

Посилання на репозиторій: <https://github.com/Ellviraaaaaa/GraphicEditor.git>

Висновок

У даній лабораторній роботі я реалізувала патерн проектування Prototype для графічного редактора, що дозволило оптимізувати процес створення нових об'єктів шляхом їхнього клонування. Завдяки цьому шаблону вдалося значно скоротити час ініціалізації складних об'єктів, таких як палітри кольорів, стилі шрифтів і фільтри, забезпечивши незалежність копій від оригінальних екземплярів.

Застосування Prototype дозволило зменшити дублювання коду та спростити процес налаштування нових об'єктів у системі. Це забезпечило гнучкість архітектури програми, її продуктивність та масштабованість, а також спростило реалізацію складних функцій роботи з графічними елементами.